

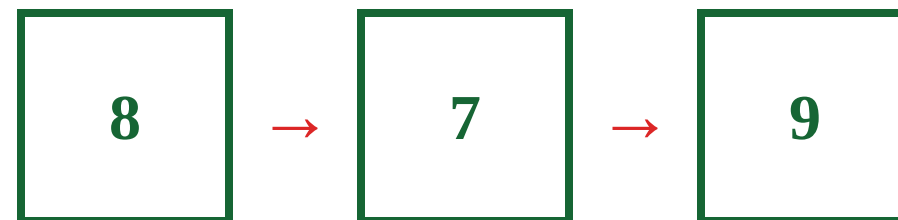
AULA DE LABORATÓRIO

Estruturas de Dados

Do valor único ao App da Turma



IFMA • JavaScript e TypeScript



NOSSO CONTEXTO

App da Turma



A mesma história vai crescer com cada estrutura.

PASSO 1

O computador guarda valores



A diagram showing three horizontal bars representing memory slots. The first and third bars are green, and the middle bar is red. Below each bar is a text value: "Ana" under the first green bar, 17 under the red bar, and true under the third green bar.

"Ana"

17

true

Começamos com um valor de cada vez.

TIPO DE DADO

Texto é string

"Ana"

STRING



PASSO 1

Quantidade é **number**

17

NUMBER



Uma idade pode ser uma quantidade.

PASSO 1

Decisão é boolean

A estudante entregou a atividade?



true

sim



false

não

Um boolean tem apenas duas possibilidades.

JavaScript infere o tipo

```
let idade = 17;
```



number

O valor ajuda a linguagem a entender o tipo.

TypeScript mostra o tipo

```
let idade: number = 17;
```

A anotação : **number** torna o tipo explícito.

Uma nota é simples

```
const nota = 8;
```



nota de uma estudante

Uma variável guarda um valor.

NOVO PROBLEMA

E se forem 30 estudantes?

```
const notaAna = 8;  
const notaBruno = 7;  
const notaCarla = 9;  
...  
const nota30 = 6;
```

Essa solução **escala**?

Cada estudante cria mais uma variável.



SOLUÇÃO

Uma coleção resolve

```
const notas = [8, 7, 9];
```

8	7	9
---	---	---

Uma variável guarda vários valores relacionados.

ARRAY

Cada item tem uma posição



Em um Array, a primeira posição é o índice **0**.

PARE E PENSE

Qual é `alunos[1]`?

PREVEJA

`alunos[1]`

0	1	2
?	?	?

- Não responda ainda. Localize primeiro a posição.

REVELAÇÃO

Resposta Bruno

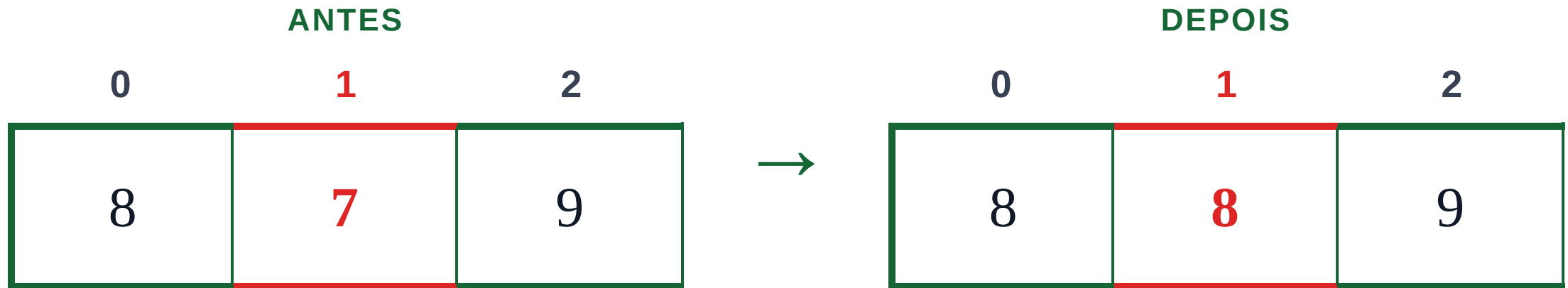


↑
`alunos[1]` retorna Bruno

ARRAY

Podemos trocar uma posição

`notas[1] = 8;`



- O índice 1 mudou. Os outros itens permanecem.

PARE E PENSE

Quantos elementos existem?

PREVEJA

`alunos.length`

0	1	2
Ana	Bruno	Carla

■ Conte os itens antes de responder.

REVELAÇÃO

Resposta: 3

0	1	2
Ana	Bruno	Carla


`alunos.length`

``length`` informa quantos elementos existem.

QUANTIDADE DE
ITENS

3

ARRAY

push adiciona no fim

`alunos.push("Diego");`

0	1	2	3
Ana	Bruno	Carla	FIM

`push()` coloca o novo item na **próxima posição**.

REVELAÇÃO

O Array ganhou um item **NOVO**

```
alunos.push("Diego");
```



`push()` acrescentou Diego **no fim**.

LISTA DE ITENS

Uma lista cotidiana

ALUNOS

Ana

Bruno

Carla



```
const alunos =  
["Ana", "Bruno",  
"Carla"];
```

- Em JavaScript, um **Array** guarda uma lista ordenada.

PARE E PENSE

Array **é** vetor?

Parecem iguais no uso.
Mas representam a mesma ideia?

USO NA LINGUAGEM

Ana · Bruno · Carla

?

MODELO DE MEMÓRIA



- Guarde a pergunta: a resposta depende do nível que estamos observando.

DISTINÇÃO IMPORTANTE

Três ideias, não uma só

Array JavaScript

LINGUAGEM

```
const alunos =  
["Ana", "Bruno"]
```

Objeto usado no código
JavaScript.

Vetor

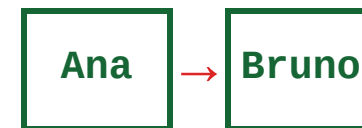
ESTRUTURA SEQUENCIAL



Posições organizadas
lado a lado.

Lista encadeada

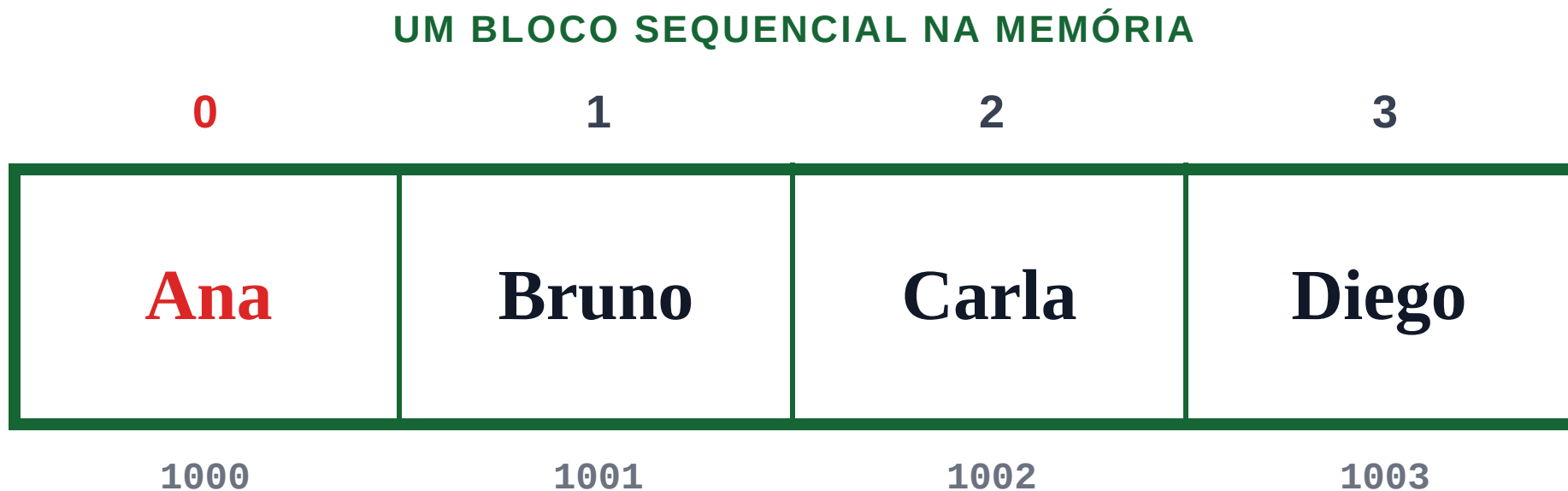
ESTRUTURA LIGADA



Nós conectados por
referências.

Não confunda: a linguagem oferece um Array; vetor e lista descrevem formas de organizar dados.

Posições lado a lado

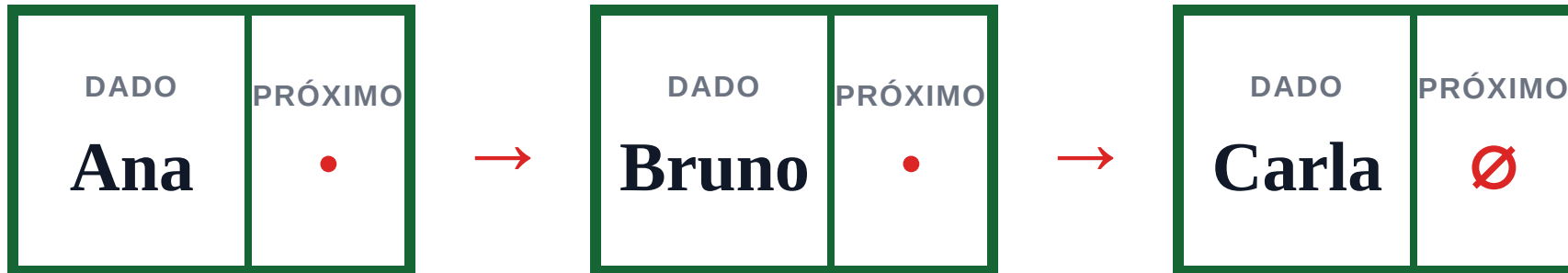


Em um vetor, os itens ocupam **posições contíguas**.

■ É um modelo de estrutura de dados; a implementação interna do JavaScript pode variar.

Nós ligados por referências

■ valor ■ referência para o próximo nó



Cada nó guarda um valor e uma **referência** para o próximo.

■ Os nós podem ocupar lugares diferentes na memória.

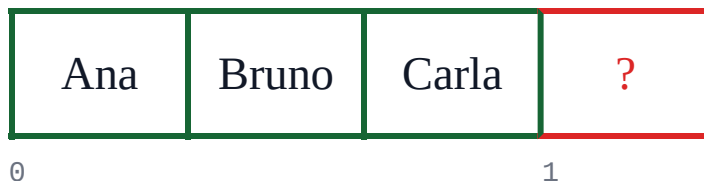
Como achar o quarto item?

Em qual estrutura
você chega ao
item **3** mais
diretamente?

Observe como os itens
estão organizados.



VECTOR



LISTA
ENCAD.



O quarto item tem índice **3**. Como cada estrutura chega até ele?

Vetor salta por índice

vetor[3]

O índice informa a posição exata.



Acesso direto: **$O(1)$**

No modelo algorítmico, a posição é calculada sem caminhar pelos itens anteriores.

Lista caminha nó a nó

Para chegar ao quarto item, começamos no início.



PERCURSO Não existe salto direto por índice: é preciso visitar os nós anteriores.

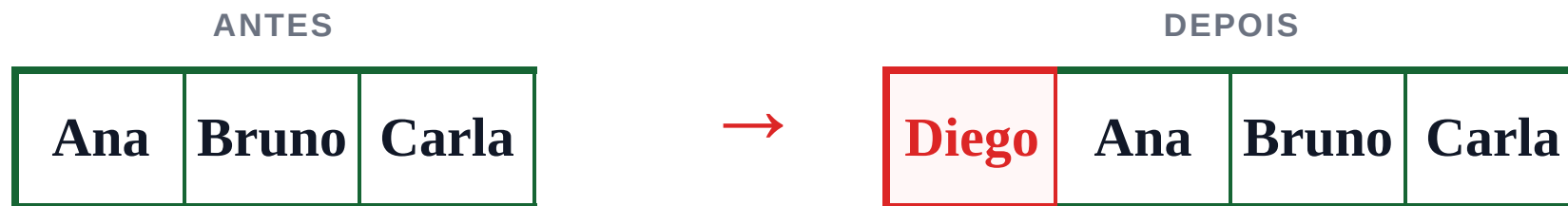
acesso por posição: **$O(n)$**

INSERÇÃO NO INÍCIO

Inserir: mover ou religar

VETOR

Abre espaço deslocando os itens.



LISTA ENCADEADA

Muda as referências entre nós.



Vetor: deslocar pode custar $O(n)$. Lista: religar custa $O(1)$ se o ponto já é conhecido.

PARE E PENSE

Qual usa mais memória?

Observe o que cada estrutura precisa guardar.

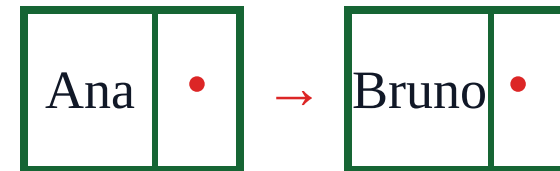
VETOR



dados + possível **capacidade**

?

LISTA ENCADEADA



dados + **referências**

- Antes da resposta, identifique o que acompanha cada dado.

REVELAÇÃO

Lista carrega **referências extras**

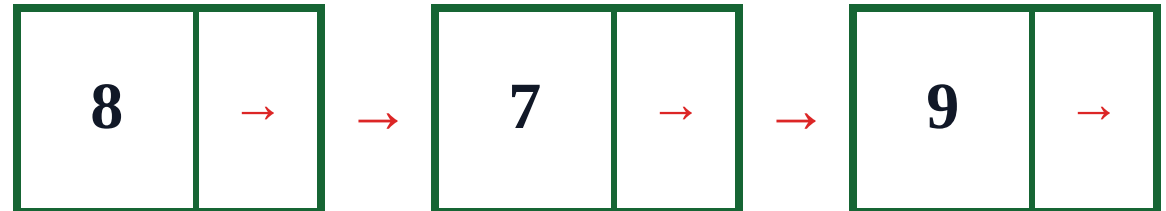
Em uma lista encadeada, cada nó precisa guardar o dado e como chegar ao próximo.

VETOR



Cada posição guarda principalmente o **valor**.

LISTA ENCADEADA



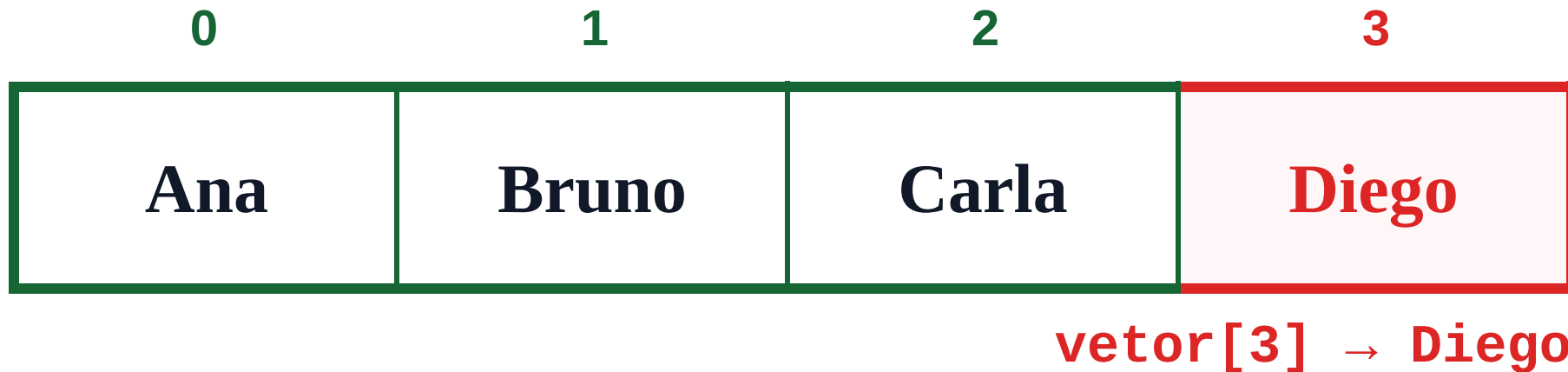
Cada nó guarda o **valor + referência** para o próximo nó.

- **Trade-off:** vetores dinâmicos podem reservar capacidade livre; listas encadeadas normalmente gastam espaço com referências. O custo real depende da implementação.

ESCOLHA DA ESTRUTURA

Use vetor quando o **índice** importa

- Escolha vetor quando você precisa chegar rapidamente a uma posição.



Acesso direto

Índices são consultados com frequência.

Leitura ordenada

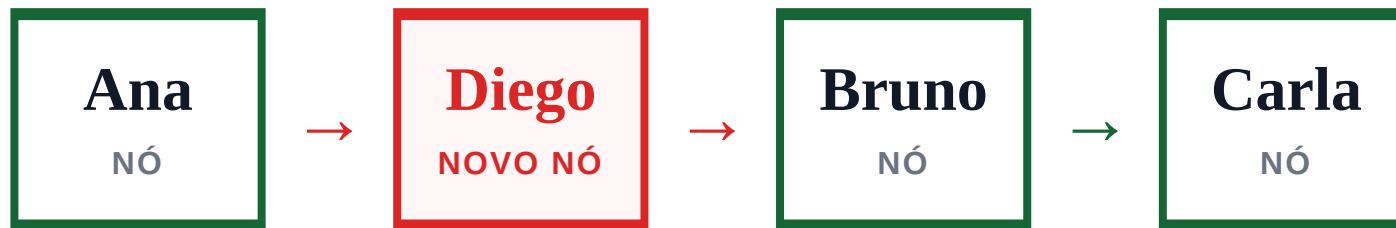
Os itens são percorridos em sequência.

Atenção

Inserir no início desloca itens.

Use lista quando **ligações** importam

INSERÇÃO LOCAL



Com o ponto de ligação conhecido, inserir Diego exige **religar referências**.

- **Muitas inserções** ou remoções locais.
- O nó anterior ou a posição de ligação já é conhecida.
- **Cuidado:** acesso por índice continua lento.

Em uma lista encadeada, a inserção pode ser **O(1)** somente quando o ponto de ligação já é conhecido.

Comece com **Array**

LISTA DE ALUNOS

```
const alunos =  
  ["Ana", "Bruno",  
   "Carla"];
```

Para listas cotidianas no código, `Array` é o ponto de partida.

- **Regra prática:** use Array para a lista; escolha a estrutura de memória apenas quando o problema e a linguagem exigirem isso.

O que você usa

Uma coleção ordenada de itens, acessada por índices.

O que não presumir

JavaScript define o comportamento do Array; o motor pode escolher sua representação interna.

NOVO PROBLEMA

Agora há linhas e colunas

O App da Turma precisa guardar notas de cada estudante em mais de um bimestre.

NOTAS DA TURMA

	B1	B2
Ana	8	9
Bruno	7	8
Carla	9	10

LINHAS: estudantes

COLUNAS: bimestres

Um único Array representa bem **linhas e colunas**?

MATRIZ

A matriz organiza duas dimensões

		COLUNAS	
		0	1
LINHAS →	linha \ coluna	0	1
	0	8	9
	1	7	8
	2	9	10

- Cada valor precisa de duas posições: **linha** e **coluna**.

PARE E PENSE

O que `notas[1][0]` encontra?

PREVEJA

`notas[1][0]`

LINHAS

COLUNAS

	0	1
0	8	9
1	?	8
2	9	10

- Primeiro escolha a **linha 1**. Depois escolha a **coluna 0**.

REVELAÇÃO

Resposta 7

notas[1][0]

		COLUNAS	
		0	1
LINHAS	0	8	9
	1	7	8
	2	9	10

LINHA 1 · COLUNA 0

7

1. Escolha a linha 1.
2. Depois a coluna 0.

TypeScript: uma matriz de números pode ser anotada como `number[][]`.

NOVO PROBLEMA

Tecnologias aparecem repetidas

TECNOLOGIAS DA TURMA

JavaScript

REPETE

TypeScript

JavaScript

REPETE

React

JavaScript

REPETE

E se eu quiser guardar cada tecnologia **uma única vez**?

Uma lista comum aceita repetições.
Agora precisamos de uma regra diferente.



Observe: JavaScript aparece três vezes, mas representa a mesma tecnologia.

SET

Set guarda cada valor **uma vez**

Quais tecnologias a turma conhece, sem repetir a mesma resposta?

ENTRADA

JavaScript
TypeScript
~~JavaScript~~
React
~~JavaScript~~



COLEÇÃO



RESULTADO

JavaScript
TypeScript
React

- Quando um valor já existe, o Set **não cria outra cópia.**

SET

Adicione um valor por vez

Primeiro criamos a coleção. Depois incluímos uma tecnologia.

1 `const tecnologias =
new Set<string>();`



Set { }

2 `tecnologias.add("JavaScript");`



Set {
JavaScript }

■ `add()` tenta inserir um valor na coleção.

PARE E PENSE

E se JavaScript entrar **de novo**?

As duas chamadas tentam adicionar exatamente o mesmo valor.

```
tecnologias.add("JavaScript");  
tecnologias.add("JavaScript");
```

QUAL É O
RESULTADO?



JavaScript

?

- O Set cria uma segunda cópia ou mantém apenas uma?

REVELAÇÃO

Set mantém um único JavaScript SEM DUPLICATA

```
tecnologias.add("JavaScript");
```

TENTATIVA
REPETIDA

~~JavaScript~~

ignorada



{

JavaScript

TypeScript

React

}

SET

Em um Set, cada valor aparece **uma única vez**.

- Use Set quando a regra do problema for **não permitir repetições**.

NOVO PROBLEMA

Quero a nota pelo nome

Em vez de procurar por posição, quero consultar diretamente o estudante.

Ana	→	8,5
-----	---	-----

Bruno	→	7,0
-------	---	-----

Carla	→	9,0
-------	---	-----

CHAVE
→ VALOR

O nome identifica a nota associada.

Qual estrutura atende esse problema?

■ **Nova necessidade:** recuperar uma informação usando uma chave, não um índice.

MAP

set guarda chave e valor

```
notasPorAluno.set("Ana", 8.5);
```

MAP: NOTAS POR ALUNO

CHAVE

Ana

→

VALOR

8.5

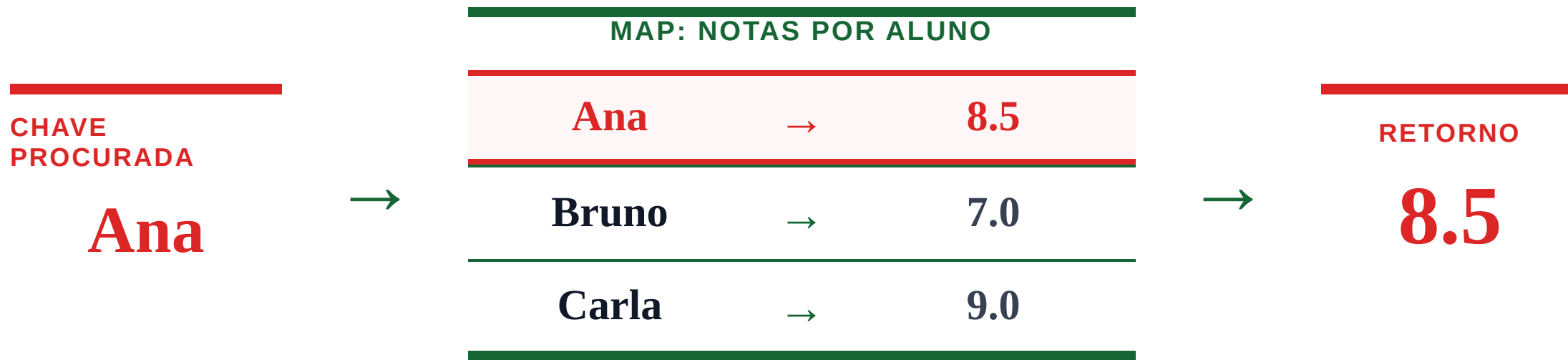
`set` cria a associação ou **atualiza** o valor dessa chave.

Fonte: MDN Web Docs — Map

MAP

get procura pela chave

```
notasPorAluno.get("Ana");
```



``get`` devolve o valor associado à **chave**.

OBJECT

Object descreve uma entidade

Uma estudante possui várias informações relacionadas.

```
const aluno = {  
  nome: "Ana",  
  idade: 16  
};
```



UMA ENTIDADE

Ana

nome : Ana

idade : 16

-
- Um **Object** reúne propriedades que descrevem uma única entidade.

Record tipo chaves previsíveis

Quando as chaves são textos e os valores seguem o mesmo tipo, `Record` descreve essa estrutura.

DECLARAÇÃO

```
const notas:
Record<string, number> = {
  Ana: 8.5,
  Bruno: 7
};
```

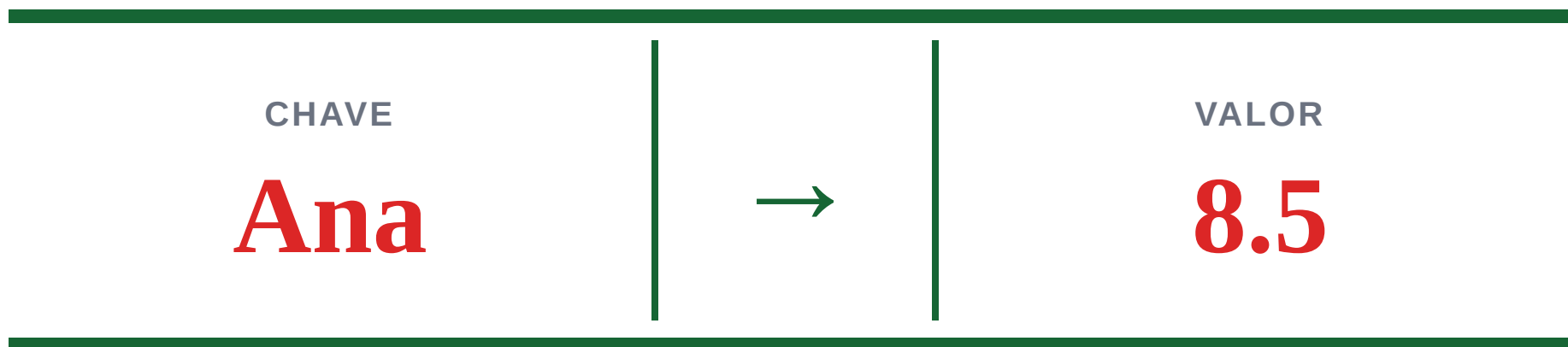
CHAVE → VALOR

Ana	→	8.5
Bruno	→	7

- **Record** descreve no TypeScript um conjunto de chaves e o tipo de valor associado a elas.

Map é chave → valor

A chave identifica a informação que queremos recuperar.



MAP: NOTAS POR ALUNO

- Cada **chave** ocorre uma vez e aponta para o valor associado. Use Map quando a pergunta for: “qual valor pertence a esta chave?”

SÍNTESE

A pergunta aponta a estrutura

COMECE PELO PROBLEMA

Preciso guardar uma **sequência**? → **Array**

Preciso de **linhas e colunas**? → **Matriz**

Não quero valores **repetidos**? → **Set**

Quero buscar por **chave**? → **Map**

A ESTRUTURA VIRA INTERFACE

APP DA TURMA

Ana

Bruno

↑
alunos: Aluno[]

- **Ideia final:** antes de escolher código, identifique que tipo de relação os seus dados precisam representar.

Referências e continuidade

Use estas fontes para revisar a sintaxe e aprofundar os modelos de estruturas de dados.

1 MDN Web Docs — Array

developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

2 TypeScript Handbook — Everyday Types

typescriptlang.org/docs/handbook/2/everyday-types.html

3 MDN Web Docs — Set

developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set

4 MDN Web Docs — Map

developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map

5 OpenDSA — Linked Lists

opensa.org/OpenDSA/Books/Catalog/html/ListLinked.html

■ **Estrutura certa** para o problema certo.

Array • Matriz • Set • Map • Object • Record • Vetor • Lista encadeada